When Can Off-the-Shelf LLMs Classify the GPU Roofline?

Anonymous Author(s)
Anonymous Institution(s)

***Abstract*—GPU kernel optimization hinges on an early decision: is a kernel limited by memory bandwidth or by compute? Answering it today requires profiling on the target GPU, hardware that is often unavailable early in development, in continuous integration, or to the AI coding agents that increasingly propose GPU code changes. We ask whether off-the-shelf large language models can make this call from static code alone, before any profiling run.**

We prompt the models to estimate a kernel's floating-point work and memory traffic, from which its Roofline arithmetic intensity and its bandwidth- versus compute-bound (BB/CB) class follow, and we evaluate 254 CUDA and OpenMP kernels across four NVIDIA GPUs. Our central finding is about evidence rather than raw accuracy: from source code, the strongest model classifies the FP32 regime correctly for roughly nine in ten kernels, but is no better than a coin flip on FP16; supplying the compiled assembly closes that gap, taking FP16 from chance to near-perfect. The signal is also cheap: a budget model triages FP32 for a fraction of a cent per query and needs no target GPU. We further characterize how stable these calls are under repeated queries. We release a dataset of kernels, build context, assembly, and profiler ground truth, and we bound the claim carefully: these models triage the Roofline regime within identifiable evidence tiers; they do not replace profiling.

***Index Terms*—software performance, GPUs, Roofline, arithmetic intensity, large language models, static performance reasoning**

I. Introduction

GPU performance engineering is fundamentally profiler-driven. Developers optimize kernels by compiling candidate variants, running them on the target accelerator, reading hardware counters, and only then deciding whether computation, memory traffic, or both deserve attention—typically through Roofline-style analysis [1]. This workflow is already costly for humans, and it becomes far more expensive in the agentic software-engineering loops that are now reshaping how GPU code is written: with the growing adoption of AI coding *teammates* [2], an LLM increasingly proposes many candidate edits and expects profiler feedback after each one [3]–[5]. Figure 1 sketches this profiler-in-the-loop workflow and the earlier, hardware-free triage step we study.

In practice, target-GPU access is often the real bottleneck. Accelerators may be rationed on a shared cluster, unavailable in early development or continuous integration, blocked by missing system dependencies, or simply too

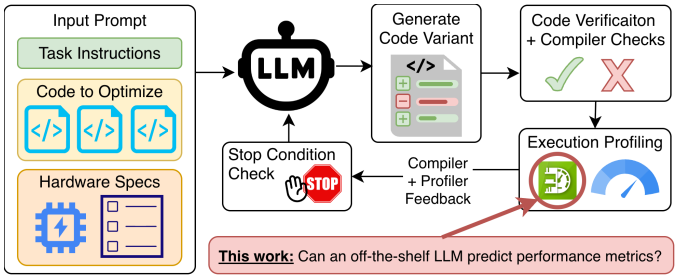

Fig. 1. Profiler-in-the-loop GPU optimization is effective, but it assumes repeated access to the target accelerator. We study whether off-the-shelf LLMs can classify a kernel's bandwidth-bound versus compute-bound Roofline regime earlier in that loop, from software artifacts alone, before runtime profiling is available.

expensive to occupy for speculative triage. The practical question is therefore not whether an LLM can *replace* a profiler, but a narrower one that a developer or coding agent faces *before* any profiling run is available: on a given target GPU, is this kernel likely to be limited by data movement or by computation?

That bandwidth-bound versus compute-bound (BB/CB) decision is the first branch of Roofline analysis [1], and it is the one that decides which optimization direction is even worth attempting. Crucially, it is a *classification*, not a precise number: a developer who learns that a kernel is firmly compute-bound can act on that without knowing its arithmetic intensity to two decimal places. We therefore make Roofline-regime classification the primary object of study, and treat the underlying Roofline Arithmetic Intensity (RAI)—the ratio of floating-point work to bytes moved—as a diagnostic that explains *why* a classification is right or wrong rather than as the headline deliverable. This reframes prior work that asked LLMs only for coarse memory-/compute-bound labels [6] or for raw metric values, and lets us report both an actionable verdict and the numeric reasoning behind it.

The central difficulty is that effective RAI is not always visible in source code. Compiler passes—lowering, instruction selection, transcendental expansion, and other low-level transformations—can change the realized compute and memory traffic in ways the source does not reveal [7]. Classifying the Roofline regime from source alone is thus a genuine test of whether an LLM can recover profiler-relevant behavior from program structure. We therefore study two evidence settings. The *source-only* setting pro-

vides only pre-compilation artifacts—source files, build commands, runtime arguments, and GPU specifications—and corresponds to early development, code review, or agent planning. The *source+SASS* setting adds the target kernel's SASS (GPU assembly) disassembly, exposing the compiler's effect to the model. Critically, SASS is obtained by *cross-compilation* (for example, `nvcc -arch=sm_90` followed by `cuobjdump -sass`) and so requires no target hardware; both settings remain execution-free at prediction time, and comparing them isolates exactly what compiler lowering adds over source-visible semantics.

Unlike prior work that fine-tunes models for GPU metric prediction [8] or studies broader GPU performance modeling [9], [10], our focus is deliberately narrow: BB/CB triage in the zero-training regime, using off-the-shelf LLMs through prompting alone. We evaluate three such models on 254 real-world CUDA and OpenMP kernels from the HeCBench suite [11] across four NVIDIA GPUs—an RTX 3080, A10, A100, and H100. For each kernel the models predict launch dimensions, per-precision FP16/FP32/FP64 floating-point work, and DRAM bytes read and written, from which we derive both RAI and its BB/CB side relative to each GPU's Roofline balance point. We highlight three findings up front: **(1)** from source artifacts alone, the strongest model already classifies the FP32 regime correctly for roughly nine in ten kernels, yet every model—frontier or budget—sits at chance for FP16; **(2)** this FP16 gap is an *evidence* gap rather than a model-capability gap: the decisive half-precision arithmetic is compiler-synthesized and absent from source, and adding cross-compiled SASS closes the gap, lifting FP16 classification from chance to near-perfect for the stronger models; **(3)** the signal is cheap and tiered—a budget model triages FP32 for a fraction of a cent per query, with no target-hardware access required—and stable enough under repeated queries to act on, while numeric RAI remains accurate only in identifiable regimes and is best used as a diagnostic, not a runtime substitute.

Research Questions. We organize the study around four guiding questions:

1) **(RQ1) Static classification:** Can off-the-shelf LLMs classify a kernel's BB/CB Roofline regime from source artifacts alone, and for which precisions?
2) **(RQ2) Compilation evidence:** What does cross-compiled SASS add to that classification, and why?
3) **(RQ3) Consistency:** How consistent are the predictions across repeated queries to the same model?
4) **(RQ4) Cost and baselines:** How do these signals compare—in accuracy and per-query cost—against lightweight static baselines, and how does that cost tier across models?

Contributions. This paper makes the following contributions:

- We formulate execution-free Roofline-regime classification as a software-engineering task for early GPU-kernel triage, with software artifacts as inputs and profiler-derived BB/CB labels as ground truth.
- We show that the source-versus-compilation evidence boundary, not model choice, governs half-precision triage, and we give the mechanistic explanation: the decisive FP16 arithmetic is compiler-synthesized and recoverable from cross-compiled SASS without target hardware.
- We characterize the run-to-run consistency of these predictions under repeated queries, identifying where a single query is trustworthy and where a simple majority vote is warranted.
- We quantify the accuracy and per-query cost of off-the-shelf prompting against context-only, deterministic, and lightweight learned static baselines, yielding a deployable cost-tiered cascade among execution-free strategies rather than an LLM-only oracle.
- We release a dataset for hardware-free Roofline reasoning—GPU kernels, compilation and execution context, post-compilation SASS, profiler-derived ground truth, and model outputs—documenting limits that future work can address with repeated-query and reasoning-versus-memorization studies.[1]

The rest of this paper is organized as follows. section II positions our work relative to Roofline analysis, execution-free GPU performance prediction, and LLM-based optimization. section III and section IV define the prediction task and study design. section V answers the research questions, and section VI and section VII translate the findings into implications and limitations for ICSE readers. section VIII concludes.

II. Related Work

Our work lies at the intersection of Roofline-guided GPU performance modeling, execution-free performance prediction, and LLM-based code reasoning. The closest prior work predicts performance without full profiler access [6], [8], [10], while adjacent areas use LLMs to generate, iteratively optimize, and reason about accelerator code. The distinction that organizes this section is our objective: we treat the profiler-derived BB/CB regime as the primary prediction target and the underlying Roofline Arithmetic Intensity (RAI) as a diagnostic, and we ask what an *off-the-shelf* model can recover from ordinary software artifacts.

Roofline modeling and static GPU performance prediction. The Roofline model is a standard framework for reasoning about whether a kernel is limited by computation or by data movement, with arithmetic intensity as one of its central quantities [1]. Long before LLMs, a substantial body of work predicted aspects of GPU performance (runtime, power, and energy) without executing the target kernel, using analytical and learned models

[1] Artifact URL omitted for double-anonymous review.

built from program structure, microarchitectural assumptions, or low-level code features. Early analytical models estimated execution time from memory- and thread-level parallelism or from compiler-derived workflow graphs that capture divergence, coalescing, and bank conflicts [12], [13], while later static approaches predicted execution time directly from PTX or other compile-time features [14]–[16]. Subsequent work extended this to GPUs with cache-aware performance, power, and energy models [17], predictors combining PTX features with random forests or XGBoost or with static estimation of execution statistics [18]–[20], bulk-synchronous and DVFS-aware analytical models [21]–[23], and recent learned predictors that transfer across workloads and devices via few-shot learning, GNN/LLM hybrids, or deep forecasters [9], [24]–[26].

These works establish the value of execution-free prediction, but they predict a different quantity (runtime, throughput, power, or energy) rather than the per-precision BB/CB regime we study, so rather than re-implement methods built for another target, we compare against purpose-built lightweight static baselines defined for our exact task (section IV). The closest static Roofline work, PolyUFC [27], does characterize CB versus BB regimes without execution, but only for CPU uncore-frequency capping on affine, polyhedral programs, and GPU static analyzers such as FlipFlop [28] target energy and launch-configuration tuning rather than arithmetic intensity. To our knowledge, no prior non-LLM method classifies an arbitrary GPU kernel as BB versus CB, or predicts its arithmetic intensity, from static source or SASS alone.

LLMs for HPC and parallel code understanding and generation. A broad line of work studies whether LLMs can better represent and generate HPC and parallel code through domain specialization, tailored tokenization, and benchmark-driven evaluation. Early domain-specific efforts include *Scope Is All You Need* [29] and *MonoCoder* [30], while HPC-Coder [31] and related work [32] show that HPC-focused corpora and fine-tuning improve performance on parallel-code tasks. Other work studies compiler-oriented representations or benchmark suites for parallel and accelerator code, including Meta LLM Compiler [33], ParEval [34], KernelBench [4], and MultiKernelBench [35], and surveys argue that HPC is a distinct setting for LLMs because correctness, hardware sensitivity, and performance portability matter simultaneously [3], [36]. Relative to this literature, our goal is not to generate kernels but to infer an interpretable, actionable performance regime from existing kernels before optimization begins.

LLMs for iterative GPU code optimization. A large neighboring literature uses LLMs to improve code performance through iterative search with compilation, execution, and profiler feedback in the loop. This includes evaluation-oriented studies such as KernelBench [4], `robust-kbench` [37], and broader assessments of LLMs on HPC optimization [38], [39], as well as feedback-driven systems such as CUDA-LLM [5], PerfCodeGen [40], SpeedGen [41], WarpDrive [42], IntelliPerf [43], Autocomp [44], SwizzlePerf [45], CudaForge [46], ParaCodex [47], SysLLMatic [48], the minimal-executable-program approach of Chu *et al.* [49], and the agent system of Wei *et al.* [50]. A related training-based line uses RL or fine-tuning to optimize GPU code, including Kevin [51], CUDA-L1 [52], KernelBlaster [53], and work integrating performance tools into model reasoning [54], and evolutionary search formulations such as LLM-GP [55] and AlphaEvolve [56] reinforce the value of iterative feedback. These systems show that LLMs can optimize code *when* hardware feedback is available; we instead ask what triage guidance is obtainable *before* it exists, so that such loops could consult an execution-free regime classifier when the accelerator is unavailable.

LLMs for GPU performance prediction. A smaller but much closer body of work uses LLMs as performance predictors rather than optimizers, and the nearest results are all *fine-tuned*. LLMPerf [10] fine-tunes models as GPU performance estimators for source programs, and Omniwise [8] introduces a fine-tuning pipeline that predicts several GPU metrics—including bandwidth, cache behavior, GFLOPs, and arithmetic intensity—directly from code, reporting over 90% of predictions within 10%. Omniwise is closest to our work, but it answers a different question (what a *trained* model predicts) and targets AMD GPUs with no released model retargetable to the NVIDIA GPUs and counters we study, so it cannot be run on our corpus; its strong fine-tuned numbers only sharpen the off-the-shelf gap we characterize. Most directly, Bolet *et al.* [6] framed LLM-based prediction as a *coarse* Roofline classification that labels whole kernels as CB or BB. We are the zero-training, classification-first counterpart to this line: we prompt off-the-shelf models and generalize that single coarse label into a per-precision decomposed signal, predicting launch dimensions, FP16/FP32/FP64 work, and DRAM bytes, from which both the BB/CB regime and RAI follow. We further contrast *source-only* reasoning with prompts containing the kernel's SASS to isolate compiler-realized work that is not syntactically obvious in source [7], [57], [58].

Program reasoning and code-property modeling. Classifying a kernel's Roofline regime also relates to whether LLMs can infer latent execution behavior from source. Work on code execution, predictive execution, and runtime-behavior reasoning shows that models can sometimes simulate program behavior but often struggle as reasoning depth and trace complexity grow [59]–[62], and benchmarks for static-analysis-style reasoning and broader code evaluation similarly find current code LLMs brittle on deeper semantic tasks [63], [64]. Related work further argues that text-only code LLMs struggle to model latent properties such as performance, motivating structured or lower-level representations [65]—which is precisely the role our *source+SASS* setting probes. Regime classification fits

this landscape because it requires recovering execution-relevant behavior from code in a form that is directly actionable for performance engineering.

Gap addressed by this work. Taken together, prior work shows that static and learned models can predict aspects of GPU performance without execution, that LLM-based optimization systems benefit strongly from profiler feedback, that fine-tuned LLMs can estimate GPU metrics, and that off-the-shelf LLMs remain imperfect at reasoning about latent program behavior. What remains underexplored is whether off-the-shelf LLMs can serve as *hardware-free BB/CB triage tools*, classifying a kernel's Roofline regime directly from ordinary software artifacts well enough to guide where scarce profiling and tuning effort should be spent. We address this gap by studying per-precision regime classification across GPU architectures, separating source-visible from post-compilation evidence, and characterizing the regimes where off-the-shelf prompting is reliable, merely useful for triage, or unreliable.

III. Task Formulation and Methodology

A. Prediction Task

Our target is the DRAM-level Roofline Arithmetic Intensity (RAI) of a GPU kernel's *first invocation*. At prediction time, the LLM does not execute the program, collect profiler counters, or observe runtime traces. Instead, it receives software artifacts and must infer the quantities from which RAI is derived.

We do not ask the model to emit RAI directly. For each profiled sample, meaning one first-invocation kernel execution on one GPU, the model predicts seven values:

1) grid size,
2) block size,
3) FP16 operation count,
4) FP32 operation count,
5) FP64 operation count,
6) DRAM bytes read, and
7) DRAM bytes written.

We then derive RAI separately for each precision:

$$\texttt{RAI}_p = \frac{\texttt{FLOP}_p}{\texttt{BytesRead} + \texttt{BytesWritten}}, \qquad p \in \{\texttt{FP16}, \texttt{FP32}, \texttt{FP64}\}$$

This decomposition makes failures easier to interpret. An incorrect launch configuration propagates into total work; an incorrect FLOP count indicates semantic or compiler-lowering errors; and an incorrect byte estimate indicates difficulty reasoning about DRAM-visible traffic.

We intentionally keep separate FP16/FP32/FP64 RAI values rather than collapsing mixed-precision kernels into a single scalar. This matches the per-precision balance points used later for triage and makes compiler-injected FP16 work visible. It also means that a kernel may have zero FP64 RAI but nonzero FP32 RAI, which we treat as a feature of the task rather than a modeling mistake.

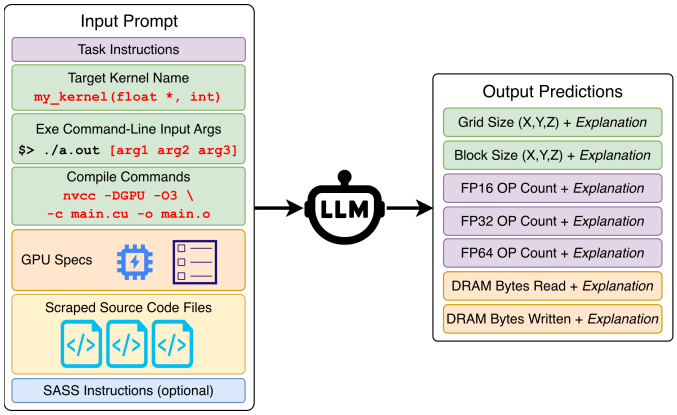

Fig. 2. Per-kernel prompting workflow. Each query includes the target kernel name, source files, compile commands, runtime arguments, and GPU specifications. The *source+SASS* setting adds target-kernel SASS sections and relevant helper routines so we can measure the value of post-compilation evidence.

B. Input Evidence

Figure 2 summarizes the evidence we provide to the model. Every prompt contains the kernel identifier (mangled and demangled), relevant source files, compile commands, runtime arguments, and target-GPU specifications. The *source+SASS* configuration augments the same prompt with SASS sections for the target kernel and helper routines that may execute on its path.

The distinction between *source-only* and *source+SASS* is important. *source-only* corresponds to early development, code review, or agent planning when only pre-compilation artifacts are available. *source+SASS* still avoids target execution and profiling at prediction time, but it assumes target-specific compilation is possible, which is a materially different stage of the workflow. We therefore treat *source+SASS* as *execution-free* rather than broadly "hardware-free."

The model returns a structured tool-call response containing the seven predicted quantities plus short explanations. Those explanations are useful for sanity checking and for the later exploratory failure analysis, but they are not used to compute the reported metrics.

C. Profiler-Derived Ground Truth

Ground truth comes from an offline benchmark-building process, not from the prediction loop itself. We profile each benchmark with NVIDIA Nsight Compute and retain the first invocation of the target kernel in each run. We repeat the executable three times and average the retained first-invocation counters to reduce profiling noise.

The exact DRAM-traffic counters are dram__bytes_read.sum and dram__bytes_write.sum. Floating-point counts are reconstructed from executed-operation counters:

- FP32:
  smsp__sass_thread_inst_executed_op_fadd_pred_on.sum.per_cycle_elapsed, smsp__sass_thread_inst_

executed_op_fmul_pred_on.sum.per_cycle_elapsed, derived__smsp__sass_thread_inst_executed_op_ffma_pred_on_x2.

- FP64:

  smsp__sass_thread_inst_executed_op_dadd_pred_on.sum.per_cycle_elapsed, smsp__sass_thread_inst_executed_op_dmul_pred_on.sum.per_cycle_elapsed, derived__smsp__sass_thread_inst_executed_op_dfma_pred_on_x2.

- FP16:

  smsp__sass_thread_inst_executed_op_hadd_pred_on.sum.per_cycle_elapsed, smsp__sass_thread_inst_executed_op_hmul_pred_on.sum.per_cycle_elapsed, derived__smsp__sass_thread_inst_executed_op_hfma_pred_on_x4.

We convert those per-cycle rates to total operation counts using smsp__cycles_elapsed.avg.per_second and gpu__time_duration.sum.

These choices matter for scope. Because bytes are counted at the DRAM interface, the task asks the model to predict *DRAM-visible* traffic rather than raw source-level memory operations. Writes that remain in the cache hierarchy until later are not immediately visible in the denominator. Likewise, compiler-injected helper routines for division or transcendental functions contribute to the measured FLOP counts even when they are not obvious in source. Those are precisely the cases where *source+SASS* may add value.

D. Why First Invocation and DRAM-Level RAI

We make two deliberate simplifications. First, we evaluate only the first invocation of each kernel, because repeated invocations introduce temporal cache reuse and cross-kernel interference that would substantially widen the reasoning space. Second, we focus on DRAM-level RAI rather than a cache-aware Roofline. This keeps the prediction target tied to one interpretable denominator while still exposing realistic sources of error such as L2 write-back behavior.

These simplifications narrow the claim. The paper studies execution-free early triage for a first kernel invocation at the DRAM level; it does not claim to predict steady-state runtime behavior, cache-resident arithmetic intensity, or end-to-end application performance.

IV. Study Design

A. Benchmark Corpus

We draw workloads from the HeCBench suite [66], focusing on CUDA and OpenMP target-offload programs because they are both common GPU programming models and represent distinct source/build ecosystems [67]. The final corpus contains 95 benchmark programs: 56 CUDA programs and 39 OpenMP programs. Those programs expose 254 first-invocation GPU kernels: 155 CUDA kernels and 99 OpenMP kernels.

Profiling the corpus with NVIDIA Nsight Compute on four NVIDIA GPUs yields 1,016 samples. Each sample induces three precision-specific RAI labels (FP16/FP32/FP64), so the profiler-derived ground-truth space contains 3,048 precision-specific evaluation points before any model-query filtering.

TABLE I
Profiled GPUs and DRAM-level balance points used for BB/CB classification. Peak throughput is in TFLOP/s; balance points (BP) are in FLOP/byte.

	RTX 3080	A10	A100	H100
Architecture	Ampere	Ampere	Ampere	Hopper
Compute cap.	8.6	8.6	8.0	9.0
BW (GB/s)	760	600	1,555	3,360
L2 (MB)	5	6	40	50
FP16 peak	30.55	15.62	77.97	133.82
FP32 peak	30.55	15.62	19.49	66.91
FP64 peak	0.477	0.244	9.75	33.45
FP16 BP	40.20	26.03	50.14	39.83
FP32 BP	40.20	26.03	12.53	19.91
FP64 BP	0.63	0.41	6.27	9.96

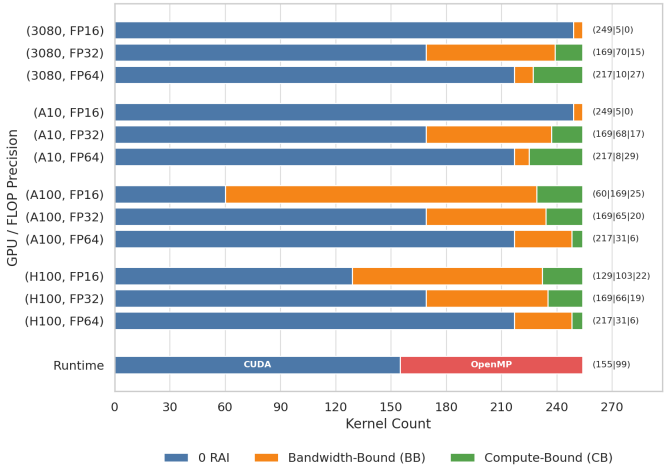

Fig. 3. Ground-truth RAI class distribution by GPU and precision. Zero-valued RAI cases dominate every precision, FP16 nonzero cases are concentrated on A100/H100, and the nonzero class mix varies by device. These imbalances motivate separate evaluation of zero detection, nonzero regression, and nonzero BB/CB triage.

We profile on four GPUs chosen to span different memory and compute regimes: RTX 3080, A10, A100, and H100. Table I lists the hardware characteristics and the resulting DRAM-level balance points used for BB/CB classification. We deliberately target DRAM-level RAI rather than a cache-aware Roofline variant, because DRAM traffic gives one profiler-derived signal that can be compared consistently across all samples. The paper's architectural question is therefore not whether models generalize to all accelerators; it is whether the same prompting strategy remains useful across noticeably different NVIDIA deployment targets.

Figure 3 highlights the resulting class imbalance. Across the full 1,016 samples, FP16 has 687 zero cases, FP32 has 676, and FP64 has 868. FP16 is especially architecture-sensitive because the A100/H100 binaries contain many more nonzero FP16 measurements than the 3080/A10.

This is precisely why we separate the evaluation tasks instead of reporting a single aggregate error number.

B. Models, Prompts, and Sample Accounting

We study three off-the-shelf LLMs: `GPT-5.4`, `Opus 4.6`, and the cheaper `GPT OSS` baseline. All reported comparisons use the two released prompt configurations that differ only by the presence or absence of SASS: *source-only* and *source+SASS*. We do not fine-tune the models, train task-specific heads, or expose runtime feedback inside the prediction loop. The committed run script invokes one trial per sample for the model identifiers `openai/gpt-5.4`, `anthropic/claude-opus-4.6`, and `openai/gpt-oss-120b`; the released OpenRouter client configuration defaults to temperature 0.2 and top-p 0.1.

The expensive frontier models were queried once per sample in the released evaluation subset. That is a methodological limitation of the artifact, so we report coverage explicitly rather than hiding it inside aggregate metrics. The completed evaluation subset contains:

- 3,048 *source+SASS* and 3,048 *source-only* predictions for `Opus 4.6`,
- 3,048 *source+SASS* and 3,045 *source-only* predictions for `GPT-5.4`, and
- 2,535 *source+SASS* and 2,574 *source-only* predictions for `GPT OSS`.

`GPT OSS` incompletions primarily reflect context-overflow or completion failures on long prompts, so we report coverage explicitly instead of folding missing samples into aggregate quality metrics. For fair cross-model comparisons, we use the 732 samples completed by all three models in both prompt settings. Within-model source-versus-SASS comparisons are paired on the subset completed by both settings for that model.

The fixup tables are generated read-only from the same artifact products used by the repository reproduction script. The direct-prompting `make_plots_for_paper.py` script reconstructs model predictions from the restored PostgreSQL dump and writes the paper-facing shared-sample outputs; the website mirror `site-data.json` is the derived prediction table used by the fixup generators. The static baselines additionally read the released `dataset-creation/gpuFLOPBench.json`, which contains source links and static instruction-mix data.

C. Evaluation Tasks and Metrics

The evaluation intentionally separates four related but distinct tasks:

a) Task A: zero vs. nonzero detection.: We first ask whether a model can distinguish kernels whose per-precision RAI is zero from kernels with nonzero RAI. Because the obvious trivial baseline is to predict zero everywhere, we report balanced accuracy rather than raw accuracy. Only an exact zero numeric prediction counts as "zero"; any positive or non-finite prediction is conservatively treated as "nonzero."

TABLE II
How to read the evaluation metrics.

Metric	Reader interpretation
$\leq$25% APE	Fraction of nonzero samples where numeric RAI is close enough to treat as an accurate static estimate.
MedAPE	Typical nonzero numeric error; useful for seeing heavy-tailed failures, not the whole story.
BalAcc	Macro recall for imbalanced triage tasks, so zero or bandwidth-heavy classes cannot dominate the score.

b) Task B: nonzero RAI regression.: For samples whose ground-truth RAI is nonzero, we report median absolute percent error. The distributions are heavy-tailed, so we emphasize medians and the supporting boxplots rather than means. As a numeric lower bound, we also compare against a trivial global-median-RAI heuristic, which produces approximately 98.5%, 99.9%, and 99.6% median APE for FP16, FP32, and FP64 respectively on the shared subset.

c) Task C: nonzero BB/CB triage.: For nonzero cases, we compare the expected and predicted side of the GPU-specific Roofline balance point. We report balanced accuracy because the class ratios are uneven. The released artifact also exports compute-bound precision/recall/F1 so reviewers can see whether a method reaches high recall by over-predicting the compute side. If a model predicts zero for a truly nonzero kernel, we count that as a memory-side failure and collapse it onto the bandwidth-bound side for this binary triage metric. This is conservative, but appropriate: a zero prediction still fails to identify compute-side behavior.

d) Task D: three-way triage.: Finally, we retain the full {zero, BB, CB } label space to show whether a model preserves the higher-level triage outcome that a developer or agent would see. We report multiclass balanced accuracy (macro recall). As in Task C, non-finite predictions are conservatively counted as memory-side failures rather than as a separate abstain class.

We include two trivial class baselines for context. Always predicting zero yields 0.50 balanced accuracy on Task A and 0.33 balanced accuracy on Task D. Always predicting the majority nonzero class (bandwidth-bound in all three precisions here) yields 0.50 balanced accuracy on Task C.

We also add three simple non-LLM reference baselines. The *context median* baseline uses grouped cross-validation and predicts the training-fold median RAI for each runtime×GPU pair, with runtime-only, GPU-only, and global fallbacks when a pair is unseen. This is a deliberately naive context-only baseline that uses no code or disassembly evidence. We also add two deterministic non-LLM reference heuristics derived from the released artifact. The *source lexical* heuristic uses only kernel-linked

TABLE III
STATIC REFERENCE BASELINES AND THEIR INPUTS.

Baseline	Inputs and training protocol
Context median	Runtime and GPU only; grouped cross-validation predicts the training-fold median RAI with runtime/GPU fallbacks.
Source lexical	Deterministic source-token counts from `gpuFLOPBench.json`: arithmetic operators, memory references, types, and math calls; no training.
SASS mnemonic	Deterministic architecture-specific instruction-mix counts from `gpuFLOPBench.json`: floating-point and global-memory mnemonics; no dynamic counts.
Learned RF/ET	Random forest or extra-trees Stage 1 zero classifier plus Stage 2 $\log(1 + RAI)$ regressor; GroupKFold by program; *source-only* uses source lexical counts plus runtime/GPU, and *source+SASS* adds static SASS instruction counts.

source files: if no precision-relevant floating-point type or math-library token appears, it predicts zero; otherwise it estimates RAI as approximate arithmetic-token count divided by four times approximate memory-reference count. The *SASS mnemonic* heuristic maps each GPU to the released architecture-specific static instruction mix, counts precision-relevant floating-point arithmetic mnemonics, counts global-memory mnemonics, and converts those counts into an approximate static RAI using 2/4/8-byte denominators for FP16/FP32/FP64. These baselines are intentionally coarse. They ignore loop trip counts, control-flow frequencies, vector widths, and dynamic access sizes, so we use them only as deterministic reference points rather than as optimized competitors.

We additionally add lightweight learned static baselines from two tree families: random forests and extra trees. For each prompt setting and family, we run a five-fold GroupKFold evaluation grouped by program. Stage 1 predicts zero versus nonzero RAI; Stage 2 regresses $\log(1 + RAI)$ on the nonzero training samples and emits zero whenever Stage 1 predicts zero. The *source-only* setting uses only source-derived lexical counts plus runtime and GPU identifiers; the *source+SASS* setting adds architecture-specific static instruction-mix counts derived from the released post-compilation artifact. The generated artifact tables report the seed-0 runs and five-seed sensitivity for the learned baselines. Because these baselines are trained and evaluated only within the released corpus, we use them as lightweight static references rather than as production-quality predictors.

Secondary checks appear only where they answer a specific comparison question. Recall@10% measures fixed-budget profiling utility, exact paired binomial tests summarize discordant Task C wins on the same samples, and Cliff's δ is reserved for exploratory feature-vote error analysis.

D. Exploratory Failure Analysis

The released artifact also contains an auxiliary feature-voting pipeline. It uses inexpensive LLMs to label source patterns such as division, special math, reusable subexpressions, and loop-invariant floating-point work. We retain that analysis because it helps characterize where prediction errors cluster, but we explicitly treat it as exploratory. The feature labels are not human-validated in the released data, so we use them to describe repeated failure patterns rather than to make causal claims.

V. EVALUATION

This section evaluates whether off-the-shelf LLMs can act as useful Roofline triage tools for GPU kernels. We organize the evaluation around the four research questions of section I: whether the models can classify a kernel's Roofline regime from static artifacts (RQ1), what cross-compiled SASS adds and why (RQ2), how consistent the predictions are across repeated queries (RQ3), and how the signals compare in accuracy and per-query cost against lightweight static baselines (RQ4). We focus on three LLMs throughout: `GPT-5.4`, `GPT OSS`, and `Opus 4.6`.

Unless stated otherwise, the cross-model comparisons use the 732 samples completed by all three models in both prompt settings; this shared subset avoids coverage bias from the incomplete `GPT OSS` runs while preserving paired *source-only*-versus-*source+SASS* comparisons. We report nonzero BB/CB regime classification as the primary outcome and treat the numeric RAI error as a diagnostic that explains *why* a classification succeeds or fails. Because the RAI-percent-error distributions are heavy-tailed, we emphasize balanced accuracy, medians, and interquartile behavior rather than means.

A. RQ1: Can LLMs Classify the Roofline Regime From Static Artifacts?

For single precision, yes; for half precision, not from source alone. Table IV reports nonzero BB/CB balanced accuracy by model, precision, and evidence setting. There are a few trends to note: **(1)** From *source-only* prompts, `Opus 4.6` classifies the FP32 regime at 0.940 balanced accuracy and FP64 at 0.755; `GPT-5.4` reaches 0.889/0.670, and even the budget `GPT OSS` reaches 0.875/0.690. These are strong numbers for a pre-compilation signal: the models recover the compute-versus-memory branch for most single-precision kernels using only source code, build context, runtime arguments, and GPU specifications. **(2)** The half-precision case is a clean failure. From source alone, every model—including the frontier ones—sits exactly at the 0.500 chance baseline for FP16. The same models that classify single-precision regimes well are no better than a coin flip on half precision when given only source; we return to why in RQ2.

Because the panel is class-imbalanced, balanced accuracy alone could mask a model that simply over-predicts the majority class. Table V shows the imbalance is real and

TABLE IV

SHARED-SUBSET SUMMARY ON THE 732 SAMPLES COMPLETED BY ALL THREE MODELS IN BOTH PROMPT SETTINGS. EACH PRECISION COLUMN REPORTS "MEDAPE / BALACC": THE MEDIAN ABSOLUTE PERCENT ERROR ON NONZERO RAI CASES AND THE BALANCED ACCURACY ON NONZERO BB/CB TRIAGE. MEDIAN COST AND TIME ARE PER-QUERY VALUES ON THE SAME SHARED SUBSET.

Model	Evidence	Samples	FP16 MedAPE / BalAcc	FP32 MedAPE / BalAcc	FP64 MedAPE / BalAcc	Median $	Median s
`GPT-5.4`	*source-only*	732	100.0 / 0.500	59.2 / 0.889	74.5 / 0.670	0.0197	11.3
`GPT-5.4`	*source+SASS*	732	84.2 / 0.870	34.9 / 0.895	35.4 / 0.737	0.0315	12.3
`GPT OSS`	*source-only*	732	100.0 / 0.500	58.9 / 0.875	62.4 / 0.690	0.0006	33.2
`GPT OSS`	*source+SASS*	732	100.0 / 0.583	61.4 / 0.865	58.4 / 0.652	0.0008	32.5
`Opus 4.6`	*source-only*	732	100.0 / 0.500	43.9 / 0.940	52.0 / 0.755	0.0679	22.5
`Opus 4.6`	*source+SASS*	732	20.4 / 0.984	18.3 / 0.967	15.3 / 0.765	0.1066	29.8

TABLE V

GROUND-TRUTH CLASS BALANCE ON THE 732-SAMPLE SHARED SUBSET USED FOR ALL PAIRED CROSS-MODEL COMPARISONS. THE IMBALANCE IS PRECISION-DEPENDENT: FP16/FP32 HAVE MORE NONZERO SAMPLES THAN FP64, BUT ALL THREE PRECISIONS REMAIN ZERO-HEAVY, WHICH IS WHY TASKS A/C/D ARE REPORTED SEPARATELY.

Precision	Zero	Nonzero BB	Nonzero CB	Zero share
FP16	513	185	34	0.701
FP32	496	186	50	0.678
FP64	623	56	53	0.851

CB-poor: of the 732 FP16 samples only 34 are nonzero compute-bound, and the corpus is zero-heavy across all precisions. Balanced accuracy is macro-averaged and so cannot be inflated by the zero- or BB-heavy classes, and the compute-bound precision/recall breakdown (Table VI) confirms the strong numbers are not an over-prediction artifact: *source-only* `Opus 4.6` reaches 1.000 compute-bound precision at 0.880 recall for FP32, and 0.966 precision at 0.528 recall for FP64. The lower FP64 recall is honest—the models miss roughly half of the compute-bound FP64 kernels even when they place the rest correctly.

The takeaway is that **(a)** off-the-shelf LLMs are already reliable single-precision regime classifiers from source alone, with a clear "you-get-what-you-paid-for" ordering from `GPT OSS` to `Opus 4.6`, and **(b)** half-precision triage is not available from source and must escalate to more evidence, which motivates RQ2.

B. RQ2: What Does Cross-Compiled SASS Add, and Why?

It closes the half-precision gap, and the mechanism is compiler-synthesized arithmetic. Adding SASS lifts FP16 BB/CB balanced accuracy from the 0.500 *source-only* baseline to 0.984 for `Opus 4.6` and 0.870 for `GPT-5.4` (Table IV). The gain is specific to the evidence, not the model: the same frontier models were at chance for FP16 with source, while `GPT OSS`—which makes weaker use of long SASS prompts—only reaches 0.583. SASS helps single precision much more modestly (`Opus 4.6` FP32 rises 0.940→0.967, FP64 0.755→0.765), because single-precision arithmetic is already mostly source-visible.

The reason is mechanistic and visible in the released samples. Half-precision throughput on these GPUs comes from packed `HFMA2` instructions that the compiler synthesizes; they are not syntactically present in source, so source-only reasoning conservatively predicts zero FP16 work. In the `accuracy-cuda` kernel on the A100, *source-only* `Opus 4.6` predicts zero FP16 work and gets the regime wrong, while with SASS it attributes the nonzero FP16 signal to an executed `HFMA2.MMA` instruction and classifies correctly. This is why the result generalizes beyond a single model: *half-precision Roofline reasoning requires compilation, not execution*, and cross-compilation supplies it without the target GPU.

Numeric RAI as a diagnostic. The numeric error tracks the same evidence boundary, which is why we read it as a diagnostic rather than a headline. Figure 4 shows the distribution of nonzero RAI absolute percent error. Where classification is strong the numeric estimate is also close: with SASS, `Opus 4.6` lands within 25% of the profiled RAI on 47.0% of nonzero FP16, 53.0% of FP32, and 44.0% of FP64 samples, with median errors of 20.4%, 18.3%, and 15.3%. Where source is blind (FP16), numeric accuracy collapses to under 1% of samples within 25%. The distributions are heavy-tailed—some kernels are predicted almost exactly while others are orders of magnitude off—so a deployed tool should treat numeric RAI as a coarse hint and rely on the more robust regime classification.

Error-prone code features. Using the source-feature labels of section III, the residual numeric errors concentrate where the same code patterns appear regardless of model or prompt type: FLOP division, transcendental ("special") math functions, reusable float subexpressions, and loop-invariant FLOP work. These features either inject FLOPs the compiler lowers from a single source token (division, transcendentals) or remove repeated work the model over-counts statically (subexpression elimination, loop-invariant hoisting), so they directly perturb the FLOP term of RAI. The practical implication is that users should expect the largest RAI errors—and should most want SASS—on kernels dense in these features.

C. RQ3: How Consistent Are the Predictions Across Repeated Queries?

LLM predictions are nondeterministic, so a single query per kernel cannot establish reliability. We resample the

TABLE VI

NONZERO COMPUTE-BOUND PRECISION/RECALL/F1 ON THE 732-SAMPLE SHARED SUBSET. THESE METRICS COMPLEMENT THE BALANCED-ACCURACY HEADLINE BY EXPOSING WHETHER A METHOD REACHES HIGH RECALL BY OVER-PREDICTING THE COMPUTE-BOUND SIDE. A DASH MARKS AN UNDEFINED VALUE: WHEN A MODEL PREDICTS NO COMPUTE-BOUND KERNELS FOR A PRECISION (AS EVERY MODEL DOES FOR *SOURCE-ONLY* FP16), RECALL IS 0 AND PRECISION/F1 HAVE A ZERO DENOMINATOR.

Model	Evidence	FP16 P/R/F1	FP32 P/R/F1	FP64 P/R/F1
`GPT-5.4`	*source-only*	– / 0.000 / –	0.909 / 0.800 / 0.851	0.950 / 0.358 / 0.521
`GPT-5.4`	*source+SASS*	0.730 / 0.794 / 0.761	0.952 / 0.800 / 0.870	0.903 / 0.528 / 0.667
`GPT OSS`	*source-only*	– / 0.000 / –	0.950 / 0.760 / 0.844	0.917 / 0.415 / 0.571
`GPT OSS`	*source+SASS*	0.750 / 0.176 / 0.286	0.949 / 0.740 / 0.831	0.900 / 0.340 / 0.493
`Opus 4.6`	*source-only*	– / 0.000 / –	1.000 / 0.880 / 0.936	0.966 / 0.528 / 0.683
`Opus 4.6`	*source+SASS*	0.850 / 1.000 / 0.919	0.979 / 0.940 / 0.959	0.938 / 0.566 / 0.706

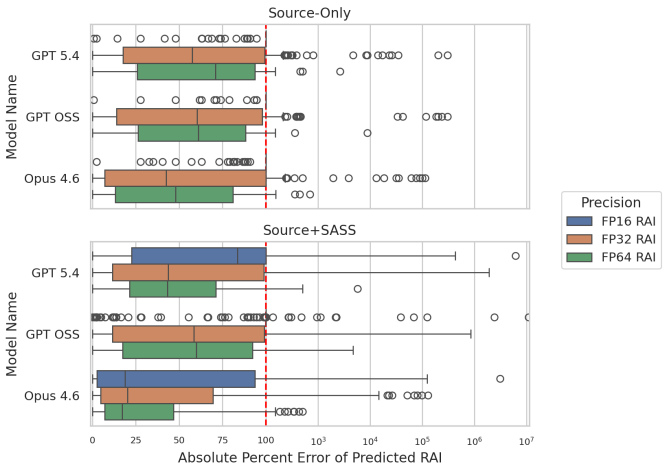

Fig. 4. Absolute percent error on nonzero RAI cases, by model, precision, and evidence setting. The distribution has both a useful low-error region and a long failure tail. *source+SASS* sharply expands the low-error FP16 region for the frontier models and tightens many FP32/FP64 distributions, mirroring the classification gains.

models on the small, stratified panel of section IV—repeating each query rather than perturbing the kernel—and measure two kinds of consistency: the coefficient of variation (CV) of the predicted RAI across repeats, and whether the BB/CB call is unanimous across repeats. Table XI reports both. At the time of writing the panel is complete for `GPT-5.4` and partially complete for `GPT OSS`; the `Opus 4.6` repeats are pending and will be added.

Two trends stand out. **(1)** *source-only* predictions are highly consistent: `GPT-5.4` holds a median FP32 CV of 0.21 and an FP64 CV of 0.04, and its BB/CB call agrees on 71–100% of cells. **(2)** Adding SASS, which improves accuracy, also *increases* run-to-run variability: `GPT-5.4`'s median CV rises to 0.87 (FP16), 0.61 (FP32), and 0.33 (FP64). This is the honest cost of the evidence that cracks half precision—once the model commits to a numeric FP16 estimate rather than a degenerate zero, that estimate varies more across repeats. The decision-level consistency is more forgiving than the numeric consistency but is not perfect: the BB/CB call still agrees on only 76–86% of *source+SASS* cells, meaning roughly one in four to one in five FP32 calls can flip across identical repeated queries.

The takeaway is that single-shot use is risky for borderline kernels: a deployed triage tool should issue a small number of repeated queries and take the majority BB/CB vote, especially in the *source+SASS* setting where numeric RAI is least stable.

D. RQ4: How Do the Signals Compare in Accuracy and Cost?

The LLM signals form a clear cost/accuracy tier, they are inexpensive in absolute per-query terms, and—unlike profiling—they need no target-hardware access.

Cost tiering. Table IV reports median per-query cost, and the configurations form a tier. The budget `GPT OSS` classifies the *source-only* FP32 regime at 0.875 balanced accuracy for a median $0.0006 per query; `GPT-5.4` reaches 0.889 for $0.0197; and `Opus 4.6` reaches 0.940 for $0.0679. For FP16, only the SASS-backed frontier tier works, at $0.0315 (`GPT-5.4`) to $0.1066 (`Opus 4.6`) per query. The recurring objection from performance-modeling venues—that LLM calls might cost as much as just measuring—is blunted for the *source-only* single-precision tier: at a median $0.0006 per kernel it is inexpensive in absolute terms and, unlike profiling, needs no target-hardware access at all. We do not collect end-to-end build-and-profile timings, so we make no measured time- or cost-saving claim against profiling; the advantages we establish are absolute query cost and the elimination of target-hardware access. The frontier+SASS tier that additionally cracks FP16 is the premium option, and we treat its cost honestly: it is the regime where the "just measure" objection is most defensible.

Versus static baselines. A credible comparison against lightweight static and learned predictors is the most direct answer to the "how does this compare to prior static analysis?" question, and is needed to establish that the LLM signal is worth its cost beyond the profiling comparison. We are constructing a like-for-like static baseline for BB/CB classification and defer that comparison to a subsequent revision.

The takeaway across the four questions is a cost-tiered deployment rather than an LLM-only oracle: cheap models give useful *source-only* single-precision triage for fractions of a cent, frontier models with cross-compiled SASS addi-

tionally recover half precision, and a few repeated queries guard the borderline calls.

VI. Discussion

a) What this enables for AI-assisted software engineering.: The strongest positive result is that off-the-shelf LLMs can classify a kernel's BB/CB Roofline regime from software artifacts alone, without ever touching the target accelerator. That is directly relevant to AI coding agents and developer-facing tools. Before spending target-GPU time, an agent can ask a narrower question than "will this be faster?": is this kernel limited by data movement or by computation on the target device, and is the available evidence enough to answer that yet? The answer is evidence-dependent. For single precision, source artifacts are already enough for a confident regime call. For half precision, source is blind and the agent must obtain cross-compiled SASS first—but that is still a hardware-free step. Numeric RAI serves as a secondary diagnostic: when it is also accurate, it explains and reinforces the regime call; when it is in the heavy-tailed failure region, it signals that the sample should be escalated to profiling.

b) Where source-only and source+SASS fit in a real workflow.: The paper also clarifies that execution-free reasoning has two meaningful stages. *source-only* fits pre-compilation tasks such as code review, design exploration, and agent planning. *source+SASS* fits a later stage where the code can be compiled for a target architecture but the accelerator is still unavailable for execution or profiling. Treating both stages as the same "hardware-free" scenario obscures an important software-engineering distinction: compilation artifacts are themselves a development resource.

c) Cost, and why we do not claim a speedup over profiling.: A recurring objection to LLM-based performance reasoning is that API calls may cost as much as simply measuring on a GPU. Our per-query costs let us bound one side of that comparison. The source-only single-precision tier is inexpensive in absolute terms: a `GPT OSS` regime classification costs a median $0.0006 per kernel-GPU pair, so triaging the entire 732-sample shared subset costs under $0.50. The frontier+SASS tier that additionally recovers half precision is roughly $0.11 per kernel-GPU pair for `Opus 4.6`. We deliberately do not convert these figures into a speedup or dollar saving over building and profiling each kernel, because we do not collect end-to-end build-and-profile timings, and profiling wall-clock is workload- and counter-set-dependent. The advantage we do establish is categorical rather than monetary: every configuration here is execution-free and needs no target-hardware access, whereas profiling requires getting each kernel onto a contended accelerator. That access argument—not a measured cost saving—is why the method fits early development, code review, continuous integration, and agent planning, where the target GPU may be unavailable, rationed, or not yet provisioned.

TABLE VII
Deployment routing summary from the shared subset. These are corpus-backed first-stage choices, not universal rules.

Regime	First stage	Why
source-only FP16	Static-first	One-shot LLMs stay at the trivial Task C baseline; cheap source features recover more queueing signal.
source-only FP32/FP64	Frontier LLM	Best pre-compilation triage overall, especially for FP32 where paired LLM wins are decisive.
source+SASS FP16/FP32	Frontier LLM	Largest paired gains once lowered arithmetic and DRAM-visible behavior become explicit.
source+SASS FP64	Static-first, LLM optional	Cheap SASS counts and learned SASS features already match or beat one-shot LLM triage in this regime.

d) Why deterministic and learned static features still matter.: The added reference baselines sharpen the methodological boundary. A simple lexical source rule is not enough to explain the useful FP32/FP64 source-only signal, and a runtime×GPU median baseline collapses outside the zero-heavy FP16 split. But supervised source-only baselines can still recover corpus-level triage cues, and architecture-specific SASS counts already expose substantial post-compilation structure after lowering. In a few regimes, especially source-only FP16 queueing and *source+SASS* FP64 triage, the static baselines are competitive with or stronger than one-shot prompting. That motivates a *hypothesis* we do not yet validate end-to-end: a cascade that combines deterministic compiler or disassembly features, lightweight supervised models, and LLM reasoning could outperform any single strategy, routing each evidence tier to the method that wins it. Table VII records the per-regime first-stage choice the released evidence supports; we present it as a routing hypothesis, not a measured system, because we have not built or evaluated the combined cascade and because choosing a route presumes the precision and evidence tier are known in advance.

e) What the released samples look like in practice.: The released samples make the aggregate story concrete. In `accuracy-cuda`, source-only `Opus 4.6` conservatively predicts zero FP16 work because compiler-generated `HFMA2` materialization is not justified from source alone; with SASS, the same model attributes the nonzero FP16 signal to an executed `HFMA2.MMA` instruction and matches the profiler almost exactly. In `boxfilter-cuda`, the source already exposes the 21-tap loop trip count, the 64 output threads per block, and the roughly 1 MiB read footprint, so source-only FP32 reasoning is effectively exact.

f) What tool builders should and should not take from the results.: Tool builders should use the method as a triage filter, not as a ground-truth oracle. The useful deployment is to prioritize profiling effort, annotate likely memory-side versus compute-side cases, or flag kernels whose source-level reasoning is too uncertain to trust.

The unsafe deployment is to use the predictions as a runtime substitute, a speedup estimator, or a universal static analysis of GPU performance. The FP16 results make that limit especially clear: without post-compilation evidence, the models often miss work that only becomes visible after lowering.

g) What the exploratory failure map still suggests.: The generated feature-vote artifacts remain exploratory because their labels come from an auxiliary LLM-voting pipeline rather than human annotation. Their descriptive pattern is consistent with the comparative results above: division, special math, reusable subexpressions, and loop-invariant floating-point work remain difficult largely because source-visible intent is a weak proxy for compiled arithmetic and DRAM-visible traffic in exactly those cases.

h) Why the benchmark release matters.: Even with the method's limitations, the released corpus is a useful research contribution in its own right. It exposes a benchmarkable software-engineering task with source files, build context, runtime arguments, SASS, profiler-derived labels, and model outputs. The added deterministic and learned reference baselines show why that matters: the benchmark is not just for ranking off-the-shelf LLMs, but for comparing multiple execution-free reasoning strategies that win in different regimes. It still supports several follow-on directions that require new experiments: stronger compiler-grade analyzers, repeated-query robustness studies, and hybrid predictors that combine compiler features with LLM reasoning.

i) Why the claims are intentionally narrow.: The measured evidence supports execution-free triage, not general GPU-performance prediction. That narrower claim is still valuable: it better matches the observed behavior, it better fits the needs of AI-assisted software engineering, and it gives future work a cleaner benchmark target.

VII. Threats to Validity and Limitations

a) Benchmark representativeness.: The corpus comes from HeCBench and covers only the 95 benchmark programs that yielded the released 254-kernel evaluation subset. That is a meaningful but still selective sample of CUDA and OpenMP GPU software. The results should therefore be read as evidence about this benchmarked task, not as a claim about all GPU kernels or all accelerator software.

b) Hardware scope.: We study only NVIDIA GPUs, and only four of them: RTX 3080, A10, A100, and H100. The architectural generalization claims are therefore limited to those device families and their balance-point regimes. The benchmark does not cover AMD or Intel GPUs, CPU-only kernels, or other accelerator programming models such as HIP and SYCL.

c) Task scope.: The prediction target is deliberately narrow. We predict only the first kernel invocation, only DRAM-level RAI, and only the per-precision FP16/FP32/FP64 decomposition used in this study. We do not predict runtime, throughput, speedup, cache-aware arithmetic intensity, or end-to-end application performance. Readers should therefore resist extrapolating the results beyond early first-invocation triage.

d) Ground-truth measurement.: Ground truth comes from profiler counters, not from source-level reasoning. That is the right choice for our task, but it means the denominator is DRAM-visible traffic rather than all source-visible loads and stores. Cache write-back behavior, replay effects, and profiler noise can therefore influence the target quantity. We reduce noise by averaging three first-invocation profiles, but this does not eliminate all measurement uncertainty.

e) Prompting and model stability.: The released evaluation subset queries the expensive models once per completed sample. We therefore cannot estimate run-to-run stochastic variability from the existing artifact, and closed-model behavior may drift across snapshots. We address this by narrowing claims and by reporting completed-sample counts explicitly, but we cannot retroactively add repeated-query evidence.

f) Coverage imbalance.: `GPT OSS` does not complete the full evaluation subset: it has 858 source-only and 845 *source+SASS* completed samples instead of 1,016. The missing samples are primarily context-overflow or completion failures on long prompts, which is part of the deployment cost of cheaper long-context configurations. Cross-model comparisons are therefore restricted to the 732 samples completed by all three models in both prompt settings. This is a fairer comparison, but it also means that the cheapest model's reported quality is conditioned on the samples where it completed.

g) Baseline limitations.: We now add context-only, deterministic, and lightweight learned tree baselines, and the learned baselines remain stable across five seeds. But those baselines are still deliberately modest. The deterministic baselines use approximate lexical and instruction-mix counts rather than compiler-grade analyzers. The learned baselines are evaluated with grouped cross-validation on the released corpus rather than on a separately curated external training/test split. Stronger compiler-driven or larger-scale learned baselines could therefore do better than the references reported here.

h) Failure-analysis validity.: The feature/error analysis is exploratory because its source-feature labels come from an auxiliary LLM-voting pipeline, not human annotation. We therefore treat the appendix heatmap and its prevalence/BH-corrected sanity check only as descriptive evidence about recurring error patterns. It should not be interpreted as statistically validated causal evidence.

i) Possible benchmark leakage.: Because the models are off-the-shelf and trained on large web corpora, some kernels or their close variants may have appeared in pretraining data. We cannot rule out such contamination completely. That is another reason to frame the result as

an empirical benchmark study rather than a claim about intrinsic reasoning ability in isolation.

VIII. Conclusion

We framed LLM-based GPU performance reasoning as an ICSE problem: deciding a kernel's BB/CB Roofline regime from software artifacts, before any profiling run, for developers and coding agents that cannot reach the target accelerator. We can now answer the three research questions directly.

RQ1 (source-only classification). Off-the-shelf LLMs classify the single-precision Roofline regime well from source alone—`Opus 4.6` reaches 0.940 balanced accuracy for FP32 and 0.755 for FP64, and even the budget `GPT OSS` reaches 0.875 for FP32—but they are no better than chance for half precision (0.500 for all three models).

RQ2 (what SASS adds, and why). Cross-compiled SASS closes the half-precision gap, lifting FP16 balanced accuracy from 0.500 to 0.984 (`Opus 4.6`) and 0.870 (`GPT-5.4`). The cause is mechanistic: the decisive FP16 arithmetic is compiler-synthesized `HFMA2` work that is invisible in source. The actionable form of this finding is that half-precision Roofline reasoning requires *compilation*, not execution, and compilation needs no target hardware.

RQ3 (accuracy and cost). The LLM signals beat lightweight static baselines in most regimes and form a clear cost tier: source-only single-precision classification is near-free ($0.0006 per kernel) and hardware-free, while the frontier+SASS tier costs about $0.11 per kernel and is justified by access and timing rather than dollar savings. Static baselines win in source-only FP16 and *source+SASS* FP64, which is why we frame deployment as a cost-tiered cascade rather than an LLM-only oracle.

The evidence does not support a stronger claim. These models do not replace profilers, they do not predict runtime in general, numeric RAI is reliable only in identifiable regimes, and the cascade remains a routing hypothesis we have not yet validated end-to-end. What we show is narrower and useful: execution-free Roofline-regime classification is a benchmarkable software-engineering task, off-the-shelf prompting already solves it well in identifiable, cheaply-detectable evidence tiers, and the source-versus-compilation boundary—not model choice—is what governs the hardest case.

References

[1] S. Williams, A. Waterman, and D. A. Patterson, "Roofline: an insightful visual performance model for multicore architectures," *Communications of The ACM*, vol. 52, no. 4, pp. 65–76, Apr. 2009, mAG ID: 2002555321.

[2] H. Li, H. Zhang, and A. E. Hassan, "The Rise of AI Teammates in Software Engineering (SE) 3.0: How Autonomous Coding Agents Are Reshaping Software Engineering," Jul. 2025, arXiv:2507.15003 [cs]. [Online]. Available: http://arxiv.org/abs/2507.15003

[3] J. Gong, V. Voskanyan, P. Brookes, F. Wu, W. Jie, J. Xu, R. Giavrimis, M. Basios, L. Kanthan, and Z. Wang, "Language Models for Code Optimization: Survey, Challenges and Future Directions," Jan. 2025, arXiv:2501.01277 [cs]. [Online]. Available: http://arxiv.org/abs/2501.01277

[4] A. Ouyang, S. Guo, S. Arora, A. L. Zhang, W. Hu, C. Ré, and A. Mirhoseini, "KernelBench: Can LLMs Write Efficient GPU Kernels?" Feb. 2025, arXiv:2502.10517 [cs]. [Online]. Available: http://arxiv.org/abs/2502.10517

[5] W. Chen, J. Zhu, Q. Fan, Y. Ma, and A. Zou, "CUDA-LLM: LLMs Can Write Efficient CUDA Kernels," Jun. 2025, arXiv:2506.09092 [cs]. [Online]. Available: http://arxiv.org/abs/2506.09092

[6] G. Bolet, G. Georgakoudis, H. Menon, K. Parasyris, N. Hasabnis, H. Estes, K. Cameron, and G. Oren, "Can Large Language Models Predict Parallel Code Performance?" in *Proceedings of the 34th International Symposium on High-Performance Parallel and Distributed Computing*. University of Notre Dame Conference Facilities Notre Dame IN USA: ACM, Jul. 2025, pp. 1–6. [Online]. Available: https://dl.acm.org/doi/10.1145/3731545.3743645

[7] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Pearson, 2007. [Online]. Available: https://www.pearson.com/en-us/subject-catalog/p/compilers-principles-techniques-and-tools/P200000003472

[8] Z. Wang, C. Ramos, M. A. Awad, and K. Lowery, "Omniwise: Predicting GPU Kernels Performance with LLMs," Jun. 2025, arXiv:2506.20886 [cs]. [Online]. Available: http://arxiv.org/abs/2506.20886

[9] S. Lee, A. Phanishayee, and D. Mahajan, "Forecasting GPU Performance for Deep Learning Training and Inference," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, ser. ASPLOS '25. New York, NY, USA: Association for Computing Machinery, Mar. 2025, pp. 493–508. [Online]. Available: https://dl.acm.org/doi/10.1145/3669940.3707265

[10] M.-K. Nguyen-Nhat, H. D. N. Do, H. T. Le, and T. T. Dao, "LLMPerf: GPU Performance Modeling meets Large Language Models," in *2024 32nd International Conference on Modeling, Analysis and Simulation of Computer and Telecommunication Systems (MASCOTS)*. Piscataway, NJ, USA: IEEE, Oct. 2024, pp. 1–8. [Online]. Available: https://ieeexplore.ieee.org/document/10786558/

[11] HeCBench Developers, "HeCBench/ at master · zjin-lcf/HeCBench," 2026. [Online]. Available: https://github.com/zjin-lcf/HeCBench

[12] S. Hong and H. Kim, "An analytical model for a GPU architecture with memory-level and thread-level parallelism awareness," *SIGARCH Comput. Archit. News*, vol. 37, no. 3, pp. 152–163, Jun. 2009. [Online]. Available: https://dl.acm.org/doi/10.1145/1555815.1555775

[13] S. S. Baghsorkhi, M. Delahaye, S. J. Patel, W. D. Gropp, and W.-m. W. Hwu, "An adaptive performance modeling tool for GPU architectures," in *Proceedings of the 15th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, ser. PPoPP '10. New York, NY, USA: Association for Computing Machinery, Jan. 2010, pp. 105–114. [Online]. Available: https://dl.acm.org/doi/10.1145/1693453.1693470

[14] R. Lim, B. Norris, and A. Malony, "Autotuning GPU Kernels via Static and Predictive Analysis," in *2017 46th International Conference on Parallel Processing (ICPP)*, Aug. 2017, pp. 523–532, iSSN: 2332-5690. [Online]. Available: https://ieeexplore.ieee.org/document/8025326/

[15] G. Alavani, K. Varma, and S. Sarkar, "Predicting Execution Time of CUDA Kernel Using Static Analysis," in *2018 IEEE Intl Conf on Parallel & Distributed Processing with Applications, Ubiquitous Computing & Communications, Big Data & Cloud Computing, Social Computing & Networking, Sustainable Computing & Communications (ISPA/IUCC/BDCloud/SocialCom/SustainCom)*. Pis-

cataway, NJ, USA: IEEE, Dec. 2018, pp. 948–955. [Online]. Available: https://ieeexplore.ieee.org/document/8672234/

[16] J. Guerreiro, A. Ilic, N. Roma, and P. Tomás, "GPU Static Modeling Using PTX and Deep Structured Learning," *IEEE Access*, vol. 7, pp. 159 150–159 161, 2019. [Online]. Available: https://ieeexplore.ieee.org/document/8890640/

[17] A. Lopes, F. Pratas, L. Sousa, and A. Ilic, "Exploring GPU performance, power and energy-efficiency bounds with Cache-aware Roofline Modeling," in *2017 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, Apr. 2017, pp. 259–268. [Online]. Available: https://ieeexplore.ieee.org/document/7975297/

[18] L. Braun, S. Nikas, C. Song, V. Heuveline, and H. Fröning, "A Simple Model for Portable and Fast Prediction of Execution Time and Power Consumption of GPU Kernels," *ACM Trans. Archit. Code Optim.*, vol. 18, no. 1, pp. 7:1–7:25, Dec. 2021. [Online]. Available: https://dl.acm.org/doi/10.1145/3431731

[19] Y. Emonds, L. Braun, and H. Fröning, "CUDAsap: Statically-Determined Execution Statistics as Alternative to Execution-Based Profiling," in *2023 IEEE/ACM 23rd International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*. Piscataway, NJ, USA: IEEE, May 2023, pp. 119–130. [Online]. Available: https://ieeexplore.ieee.org/document/10171571/

[20] H.-Q. Tran, V.-S. Pham, T.-S. Pham, T.-D. Chu, A.-T. Mai, X. T. Nguyen, and T.-T. Dao, "Accurate Latency Predictor for Triton-Based GPU Kernels with PTX Features and XGBoost," in *2025 RIVF International Conference on Computing and Communication Technologies (RIVF)*. Piscataway, NJ, USA: IEEE, Dec. 2025, pp. 980–985, iSSN: 2473-0130. [Online]. Available: https://ieeexplore.ieee.org/document/11365185/

[21] M. Amarís, R. Y. de Camargo, M. Dyab, A. Goldman, and D. Trystram, "A comparison of GPU execution time prediction using machine learning and analytical modeling," in *2016 IEEE 15th International Symposium on Network Computing and Applications (NCA)*, Oct. 2016, pp. 326–333. [Online]. Available: https://ieeexplore.ieee.org/document/7778637/

[22] Q. Wang and X. Chu, "GPGPU Performance Estimation With Core and Memory Frequency Scaling," *IEEE Transactions on Parallel and Distributed Systems*, vol. 31, no. 12, pp. 2865–2881, Dec. 2020. [Online]. Available: https://ieeexplore.ieee.org/document/9124659/

[23] J. Lee, Y. Ha, S. Lee, J. Woo, J. Lee, H. Jang, and Y. Kim, "GCoM: a detailed GPU core model for accurate analytical modeling of modern GPUs," in *Proceedings of the 49th Annual International Symposium on Computer Architecture*, ser. ISCA '22. New York, NY, USA: Association for Computing Machinery, Jun. 2022, pp. 424–436. [Online]. Available: https://dl.acm.org/doi/10.1145/3470496.3527384

[24] A. Dey, A. Dhakal, T. Z. Islam, J.-S. Yeom, T. Patki, D. Nichols, A. Movsesyan, and A. Bhatele, "Relative Performance Prediction Using Few-Shot Learning," in *2024 IEEE 48th Annual Computers, Software, and Applications Conference (COMPSAC)*. Osaka, Japan: IEEE, Jul. 2024, pp. 1764–1769. [Online]. Available: https://ieeexplore.ieee.org/document/10633344/

[25] K. P. Selvam, P. M. Phothilimthana, S. Abu-El-Haija, B. Perozzi, and M. Brorsson, "Can LLMs Enhance Performance Prediction for Deep Learning Models?" Oct. 2024. [Online]. Available: https://openreview.net/forum?id=Txxz9fBPcJ

[26] K. Panner Selvam, "Performance Prediction Models for Deep Learning: A Graph Neural Network and Large Language Model Approach," 2025. [Online]. Available: https://orbilu.uni.lu/handle/10993/64557

[27] N. R. Shah, M. V. V. S. M. Kumar, D. Baxi, and R. Upadrasta, "PolyUFC: Polyhedral Compilation Meets Roofline Analysis for Uncore Frequency Capping," in *Proceedings of the 2026 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, 2026. [Online]. Available: https://ieeexplore.ieee.org/document/11395211/

[28] S. Rajput, A. Brandt, V. Elisseev, and T. Sharma, "FlipFlop: A Static Analysis-based Energy Optimization Framework for GPU Kernels," in *Proceedings of the 48th International Conference on Software Engineering (ICSE)*, 2026.

[29] Tal Kadosh, Niranjan Hasabnis, Vy Vo, Nadav Schneider, Neva Krien, Abdul Wasay, Nesreen K. Ahmed, Ted Willke, Gil Tamir, Yuval Pinter, Timothy G. Mattson, and Gal Oren, "Scope is all you need: Transforming LLMs for HPC Code," *arXiv.org*, Aug. 2023, aRXIV_ID: 2308.09440 MAG ID: 4386043885 S2ID: d5bf55f810cf66d3e70155bca12420d38d0601ad.

[30] Tal Kadosh, N. Hasabnis, Vy A. Vo, Nadav Schneider, Neva Krien, Mihai Capota, Abdul Wasay, Nesreen K. Ahmed, Ted Willke, G. Tamir, Yuval Pinter, Timothy Mattson, and Gal Oren, "MonoCoder: Domain-Specific Code Language Model for HPC Codes and Tasks," *IEEE Conference on High Performance Extreme Computing*, 2023, aRXIV_ID: 2312.13322 S2ID: f8c50d9bc22dea85bddf1859df7b11133a4f1c79.

[31] D. Nichols, A. Marathe, H. Menon, T. Gamblin, and A. Bhatele, "HPC-Coder: Modeling Parallel Programs using Large Language Models," in *ISC High Performance 2024 Research Paper Proceedings (39th International Conference)*, May 2024, pp. 1–12. [Online]. Available: https://ieeexplore.ieee.org/document/10528929

[32] Daniel Nichols, Aniruddha Marathe, Harshitha Menon, T. Gamblin, and A. Bhatele, "Modeling Parallel Programs using Large Language Models," *Information Security Conference*, 2023, aRXIV_ID: 2306.17281 S2ID: 6ed21424a6ab0a5b0693adf36c607dbe78b5cf5d.

[33] C. Cummins, V. Seeker, D. Grubisic, B. Roziere, J. Gehring, G. Synnaeve, and H. Leather, "Meta Large Language Model Compiler: Foundation Models of Compiler Optimization," Jun. 2024, arXiv:2407.02524 [cs]. [Online]. Available: http://arxiv.org/abs/2407.02524

[34] D. Nichols, J. H. Davis, Z. Xie, A. Rajaram, and A. Bhatele, "Can Large Language Models Write Parallel Code?" in *Proceedings of the 33rd International Symposium on High-Performance Parallel and Distributed Computing*, ser. HPDC '24. New York, NY, USA: Association for Computing Machinery, Aug. 2024, pp. 281–294. [Online]. Available: https://dl.acm.org/doi/10.1145/3625549.3658689

[35] Z. Wen, Y. Zhang, Z. Li, Z. Liu, L. Xie, and T. Zhang, "MultiKernelBench: A Multi-Platform Benchmark for Kernel Generation," Jul. 2025, arXiv:2507.17773 [cs]. [Online]. Available: http://arxiv.org/abs/2507.17773

[36] L. Chen, N. K. Ahmed, A. Dutta, A. Bhattacharjee, S. Yu, Q. I. Mahmud, W. Abebe, H. Phan, A. Sarkar, B. Butler, N. Hasabnis, G. Oren, V. A. Vo, J. P. Munoz, T. L. Willke, T. Mattson, and A. Jannesari, "The Landscape and Challenges of HPC Research and LLMs," Feb. 2024, arXiv:2402.02018 [cs]. [Online]. Available: http://arxiv.org/abs/2402.02018

[37] R. T. Lange, Q. Sun, A. Prasad, M. Faldor, Y. Tang, and D. Ha, "Towards Robust Agentic CUDA Kernel Benchmarking, Verification, and Optimization," Sep. 2025, arXiv:2509.14279 [cs]. [Online]. Available: http://arxiv.org/abs/2509.14279

[38] Bowen Cui, Tejas Ramesh, Oscar Hernandez, and Keren Zhou, "Do Large Language Models Understand Performance Optimization?" 2025. [Online]. Available: https://arxiv.org/abs/2503.13772

[39] B. Cui, T. Ramesh, O. Hernandez, and K. Zhou, "Comprehensive Evaluation of LLMs in HPC Code Performance Optimization," in *Workshop Proceedings of the 54th International Conference on Parallel Processing*, ser. ICPP Workshops '25. New York, NY, USA: Association for Computing Machinery, Dec. 2025, pp. 1–8. [Online]. Available: https://dl.acm.org/doi/10.1145/3750720.3757280

[40] Y. Peng, A. D. Gotmare, M. R. Lyu, C. Xiong, S. Savarese, and D. Sahoo, "PerfCodeGen: Improving Performance of LLM Generated Code with Execution Feedback," in *2025 IEEE/ACM Second International Conference on AI Foundation Models and Software Engineering (Forge)*. Piscataway, NJ, USA: IEEE, Apr. 2025, pp. 1–13. [Online]. Available: https://ieeexplore.ieee.org/document/11052794/

[41] N. Purschke, S. Kirchner, and A. Knoll, "SpeedGen: Enhancing Code Efficiency through Large Language Model-Based Performance Optimization," in *2025 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, Mar. 2025, pp. 1–12. [Online]. Available: https://ieeexplore.ieee.org/document/10992503/

[42] S. Damani, S. K. S. Hari, M. Stephenson, and C. Kozyrakis, "WarpDrive: An Agentic Workflow for Ninja GPU Transformations," *NeurIPS: Machine Learning For Systems Workshop*, 2024. [Online]. Available: https://mlforsystems.org/assets/papers/neurips2024/paper32.pdf
[43] M. Awad, C. Ramos, and K. Lowery, "IntelliPerf: LLM-Powered Autonomous GPU Performance Engineer," Jul. 2025. [Online]. Available: https://github.com/AMDResearch/intelliperf
[44] C. Hong, S. Bhatia, A. Cheung, and Y. S. Shao, "Autocomp: A Powerful and Portable Code Optimizer for Tensor Accelerators," Nov. 2025, arXiv:2505.18574 [cs]. [Online]. Available: http://arxiv.org/abs/2505.18574
[45] A. Tschand, M. Awad, R. Swann, K. Ramakrishnan, J. Ma, K. Lowery, G. Dasika, and V. J. Reddi, "SwizzlePerf: Hardware-Aware LLMs for GPU Kernel Performance Optimization," Aug. 2025, arXiv:2508.20258 [cs]. [Online]. Available: http://arxiv.org/abs/2508.20258
[46] Z. Zhang, R. Wang, S. Li, Y. Luo, M. Hong, and C. Ding, "CudaForge: An Agent Framework with Hardware Feedback for CUDA Kernel Optimization," Nov. 2025, arXiv:2511.01884 [cs]. [Online]. Available: http://arxiv.org/abs/2511.01884
[47] E. Kaplan, T. Bitan, L. Ghrayeb, L. Chen, T. Yotam, N. Hasabnis, and G. Oren, "ParaCodex: A Profiling-Guided Autonomous Coding Agent for Reliable Parallel Code Generation and Translation," Jan. 2026, arXiv:2601.04327 [cs]. [Online]. Available: http://arxiv.org/abs/2601.04327
[48] H. Peng, A. Gupte, R. Hasler, N. J. Eliopoulos, C.-C. Ho, R. Mantri, L. Deng, K. Läufer, G. K. Thiruvathukal, and J. C. Davis, "SysLLMatic: Large Language Models are Software System Optimizers," Oct. 2025, arXiv:2506.01249 [cs]. [Online]. Available: http://arxiv.org/abs/2506.01249
[49] R. Chu, A. Wang, X. Bai, S. Liu, and X. Dong, "GPU Kernel Optimization Beyond Full Builds: An LLM Framework with Minimal Executable Programs," Dec. 2025, arXiv:2512.22147 [cs]. [Online]. Available: http://arxiv.org/abs/2512.22147
[50] A. Wei, A. Nie, T. S. F. X. Teixeira, R. Yadav, W. Lee, K. Wang, and A. Aiken, "Improving Parallel Program Performance with LLM Optimizers via Agent-System Interfaces," 2024, version Number: 4. [Online]. Available: https://arxiv.org/abs/2410.15625
[51] C. Baronio, P. Marsella, B. Pan, S. Guo, and S. Alberti, "Kevin: Multi-Turn RL for Generating CUDA Kernels," Jul. 2025, arXiv:2507.11948 [cs]. [Online]. Available: http://arxiv.org/abs/2507.11948
[52] X. Li, X. Sun, A. Wang, J. Li, and C. Shum, "CUDA-L1: Improving CUDA Optimization via Contrastive Reinforcement Learning," Feb. 2026, arXiv:2507.14111 [cs]. [Online]. Available: http://arxiv.org/abs/2507.14111
[53] K. S. Dong, S. Modi, D. Nikiforov, S. Damani, E. Lin, S. K. S. Hari, and C. Kozyrakis, "KernelBlaster: Continual Cross-Task CUDA Optimization via Memory-Augmented In-Context Reinforcement Learning," Feb. 2026, arXiv:2602.14293 [cs]. [Online]. Available: http://arxiv.org/abs/2602.14293
[54] D. Nichols, K. Parasyris, C. Jekel, A. Bhatele, and H. Menon, "Integrating Performance Tools in Model Reasoning for GPU Kernel Optimization," Oct. 2025, arXiv:2510.17158 [cs]. [Online]. Available: http://arxiv.org/abs/2510.17158
[55] E. Hemberg, S. Moskal, and U.-M. O'Reilly, "Evolving Code with A Large Language Model," Jan. 2024, arXiv:2401.07102 [cs]. [Online]. Available: http://arxiv.org/abs/2401.07102
[56] A. Novikov, N. Vũ, M. Eisenberger, E. Dupont, P.-S. Huang, A. Z. Wagner, S. Shirobokov, B. Kozlovskii, F. J. R. Ruiz, A. Mehrabian, M. P. Kumar, A. See, S. Chaudhuri, G. Holland, A. Davies, S. Nowozin, P. Kohli, and M. Balog, "AlphaEvolve: A coding agent for scientific and algorithmic discovery," Jun. 2025, arXiv:2506.13131 [cs]. [Online]. Available: http://arxiv.org/abs/2506.13131
[57] IEEE Computer Society, "IEEE Standard for Floating-Point Arithmetic," *IEEE Std 754-2019 (Revision of IEEE 754-2008)*, pp. 1–84, Jul. 2019. [Online]. Available: https://ieeexplore.ieee.org/stampPDF/getPDF.jsp
[58] N. Louvet, J.-M. Muller, and A. Panhaleux, "Newton-Raphson algorithms for floating-point division using an FMA," in *ASAP 2010 - 21st IEEE International Conference on Application-specific Systems, Architectures and Processors*, Jul. 2010, pp. 200–207. [Online]. Available: https://ieeexplore.ieee.org/document/5540948/
[59] C. Liu, S. Lu, W. Chen, D. Jiang, A. Svyatkovskiy, S. Fu, N. Sundaresan, and N. Duan, "Code Execution with Pre-trained Language Models," May 2023, arXiv:2305.05383 [cs]. [Online]. Available: http://arxiv.org/abs/2305.05383
[60] C. Lyu, L. Yan, R. Xing, W. Li, Y. Samih, T. Ji, and L. Wang, "Large Language Models as Code Executors: An Exploratory Study," Oct. 2024, arXiv:2410.06667 [cs]. [Online]. Available: http://arxiv.org/abs/2410.06667
[61] Y. Li, H. Dhulipala, A. Yadavally, X. Rong, S. Wang, and T. N. Nguyen, "Blended Analysis for Predictive Execution," *Proceedings of the ACM on Software Engineering*, vol. 2, no. FSE, pp. 2987–3008, Jun. 2025. [Online]. Available: https://dl.acm.org/doi/10.1145/3729402
[62] J. Chen, Z. Pan, X. Hu, Z. Li, G. Li, and X. Xia, "Reasoning Runtime Behavior of a Program with LLM: How Far are We?" in *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, Apr. 2025, pp. 1869–1881. [Online]. Available: https://ieeexplore.ieee.org/document/11029885/
[63] D. Xie, M. Zheng, X. Liu, J. Wang, C. Wang, L. Tan, and X. Zhang, "CORE: Benchmarking LLMs Code Reasoning Capabilities through Static Analysis Tasks," Jul. 2025, arXiv:2507.05269 [cs]. [Online]. Available: http://arxiv.org/abs/2507.05269
[64] N. Jain, K. Han, A. Gu, W.-D. Li, F. Yan, T. Zhang, S. Wang, A. Solar-Lezama, K. Sen, and I. Stoica, "LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code," Jun. 2024, arXiv:2403.07974 [cs]. [Online]. Available: http://arxiv.org/abs/2403.07974
[65] S. Pyda, D. Nichols, and A. Bhatele, "The Shortcomings of Code LLMs in Modeling Code Properties," in *2025 IEEE/ACM International Workshop on Large Language Models for Code (LLM4Code)*. Piscataway, NJ, USA: IEEE, May 2025, pp. 193–199. [Online]. Available: https://ieeexplore.ieee.org/document/11028359/
[66] Z. Jin and J. S. Vetter, "A Benchmark Suite for Improving Performance Portability of the SYCL Programming Model," in *2023 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. Piscataway, NJ, USA: IEEE, Apr. 2023, pp. 325–327. [Online]. Available: https://ieeexplore.ieee.org/document/10158214/
[67] T. Kadosh, N. Hasabnis, T. Mattson, Y. Pinter, and G. Oren, "Quantifying OpenMP: Statistical Insights into Usage and Adoption," in *2023 IEEE High Performance Extreme Computing Conference (HPEC)*. Piscataway, NJ, USA: IEEE, Sep. 2023, pp. 1–7, iSSN: 2643-1971. [Online]. Available: https://ieeexplore.ieee.org/document/10363459/

Appendix A
Additional Stratified Error Plots

Appendix B
Static Reference Baselines

Appendix C
Seed Sensitivity

Appendix D
Illustrative Released Rows

Appendix E
Exploratory Failure Map

Appendix F
Repeated-Query Robustness (Planned)

The released evaluation subset queries each model once per sample, so it cannot characterize run-to-run stochastic variability. We plan to close this gap with a low-cost repeated-query study rather than re-running the full grid. On a stratified panel of roughly 50 kernels chosen to span

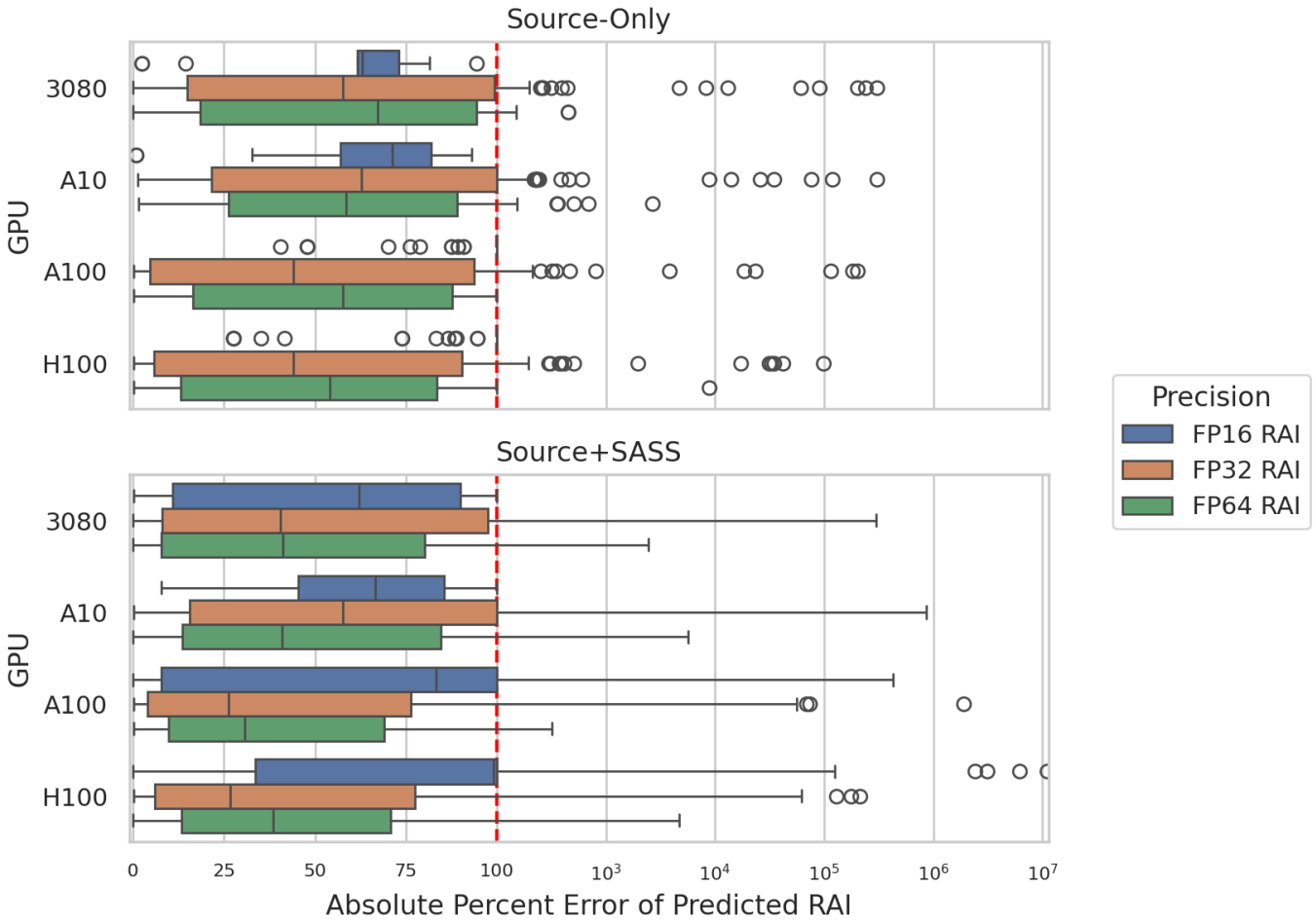

Fig. 5. Absolute percent error on nonzero RAI cases, stratified by GPU. The smaller-cache 3080/A10 settings are generally less stable than A100/H100, especially for source-level reasoning about FP32/FP64 DRAM-visible traffic.

precision, error magnitude, balance-point proximity, class, and runtime (not only low-error cases), we will query `GPT OSS` 5–10 times—and a small `Opus 4.6` subset—and report the run-to-run distribution of the headline classification metric. Table XI is the placeholder for that result; the panel-construction protocol is recorded in the project plan.

Appendix G
Reasoning versus Memorization Probe (Planned)

Because the corpus is public, some kernels may appear in pretraining data, so accurate predictions could reflect recall rather than reasoning. We plan a perturbation probe to separate the two on a panel that deliberately over-samples near-exact and well-known kernels (where memorization is the competing explanation), with high-error controls. Each kernel receives two perturbation types with opposite expectations: *cosmetic* edits (renamed identifiers, reordered functions) under which a reasoning model should be invariant, and *semantic* edits (changed loop trip counts, array dimensions, or problem-size arguments) that move the ground truth—under which a reasoning model's prediction should track the new value while a memorizing model stays anchored to the original. Where the perturbed kernel must be re-profiled to obtain new ground truth, we will use a single locally available GPU, since this probe tests the model rather than cross-device generality. We will report the correlation between predicted and true RAI change, not absolute error alone.

Appendix H
Anonymity Note

The current workspace contains public website and repository mirrors for the benchmark artifact. To preserve double anonymity, the manuscript omits those URLs. The project README records the replacement points for a de-anonymized camera-ready version.

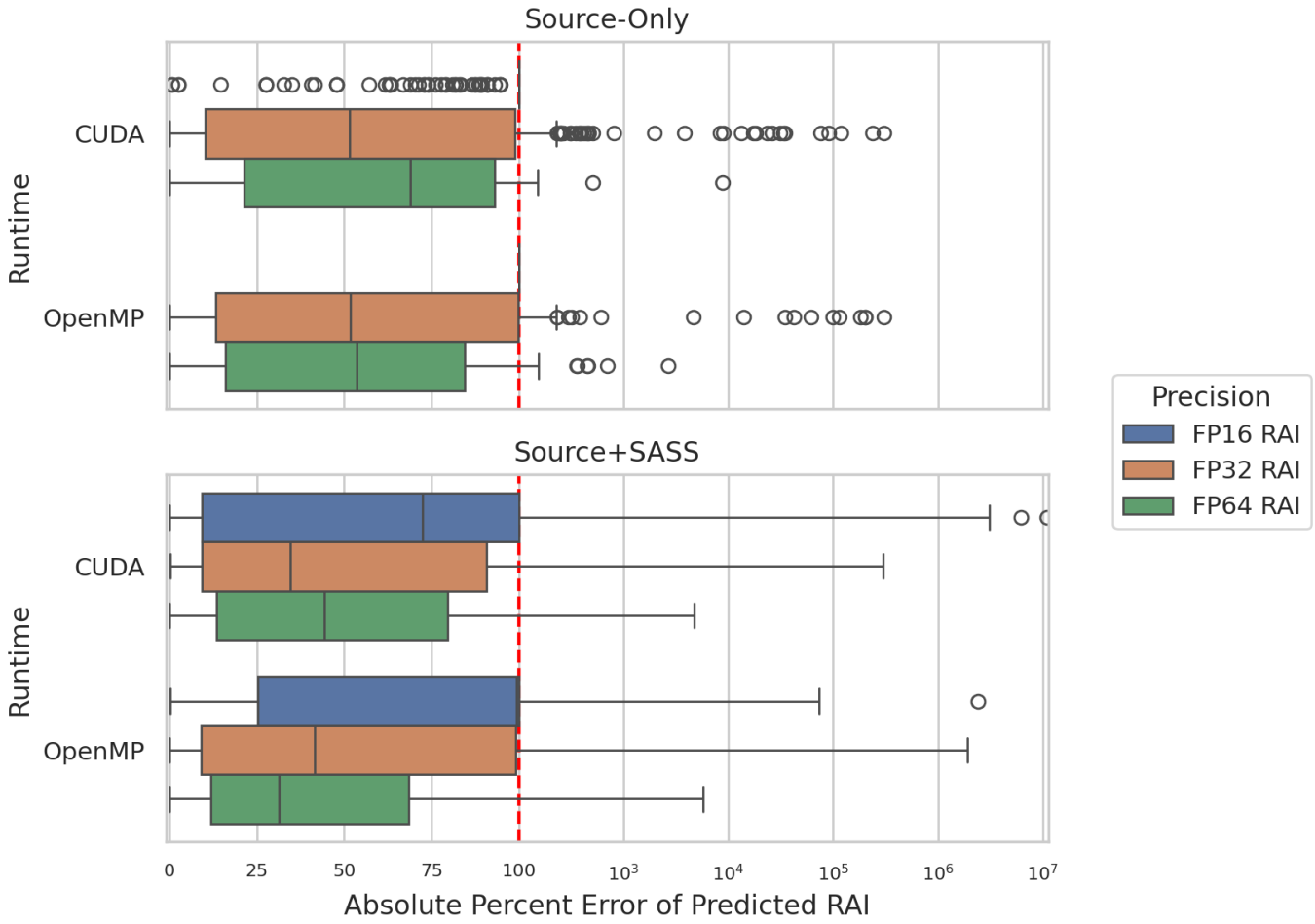

Fig. 6. Absolute percent error on nonzero RAI cases, stratified by runtime. The runtime split is precision-sensitive rather than uniform: CUDA tends to be easier for FP16/FP32 triage, while FP64 remains mixed across CUDA and OpenMP.

TABLE VIII

STATIC REFERENCE BASELINES ON THE 732-SAMPLE SHARED SUBSET. CONTEXT MEDIAN IS A GROUPED CROSS-VALIDATED RUNTIME×GPU BASELINE; LEXICAL AND MNEMONIC ARE DETERMINISTIC HEURISTICS; THE LEARNED BASELINES ARE FIVE-FOLD GROUPED TREE BASELINES GROUPED BY PROGRAM. FOR EACH PRECISION, THE SLASH-SEPARATED TRIPLE REPORTS TASK A ZERO/NONZERO BALANCED ACCURACY, TASK C NONZERO BB/CB BALANCED ACCURACY, AND TASK D THREE-WAY BALANCED ACCURACY. "MEDAPE" IS THE MEDIAN NONZERO ABSOLUTE PERCENT ERROR.

Baseline	FP16 A/C/D	FP32 A/C/D	FP64 A/C/D	FP16 MedAPE	FP32 MedAPE	FP64 MedAPE
Context median CV	0.823 / 0.500 / 0.549	0.500 / 0.500 / 0.333	0.500 / 0.500 / 0.333	100.0	100.0	100.0
Source lexical	0.523 / 0.500 / 0.351	0.693 / 0.489 / 0.444	0.651 / 0.683 / 0.531	100.0	99.7	99.5
SASS mnemonic	0.983 / 0.500 / 0.656	0.897 / 0.577 / 0.649	0.995 / 0.756 / 0.834	95.9	64.9	68.8
Source learned RF	0.845 / 0.712 / 0.699	0.804 / 0.750 / 0.674	0.659 / 0.644 / 0.544	100.0	100.0	100.0
Source learned ET	0.842 / 0.723 / 0.703	0.772 / 0.695 / 0.622	0.624 / 0.498 / 0.430	217.7	100.0	100.0
SASS learned RF	0.973 / 0.586 / 0.705	0.913 / 0.723 / 0.743	0.879 / 0.841 / 0.815	190.9	100.0	62.4
SASS learned ET	0.917 / 0.608 / 0.686	0.860 / 0.730 / 0.712	0.835 / 0.725 / 0.692	163.1	100.0	100.0

TABLE IX

SEED SENSITIVITY FOR THE LEARNED STATIC BASELINES ACROSS FIVE GROUPED-CV SEEDS. EACH ENTRY REPORTS MEAN±STD OVER SEEDS FOR TASK C BALANCED ACCURACY AND NONZERO MEDAPE. THE LOW SPREAD SHOWS THAT THE STATIC-BASELINE COMPARISONS ARE NOT DRIVEN BY AN UNUSUALLY FAVORABLE RANDOM SEED.

Baseline	FP16 C / MedAPE	FP32 C / MedAPE	FP64 C / MedAPE	FP16 std(C)	FP32 std(C)	FP64 std(C)
Source learned RF	0.731 / 102.1	0.801 / 100.0	0.621 / 100.0	0.025	0.028	0.021
Source learned ET	0.649 / 214.4	0.692 / 100.0	0.515 / 100.0	0.037	0.011	0.014
SASS learned RF	0.589 / 204.0	0.738 / 100.0	0.843 / 62.0	0.007	0.009	0.007
SASS learned ET	0.601 / 184.6	0.729 / 100.0	0.729 / 100.0	0.009	0.014	0.021

TABLE X

ILLUSTRATIVE RELEASED SAMPLES USED ONLY FOR INTUITION, NOT AS ADDITIONAL EVALUATION EVIDENCE. THESE EXAMPLES SHOW ACCURATE NUMERIC RAI PREDICTIONS IN REGIMES WHERE THE RELEVANT COMPUTATION AND MEMORY BEHAVIOR ARE VISIBLE TO THE MODEL.

Case	Released sample	Expected → predicted RAI	Why it matters
Hidden lowered FP16 work	`accuracy-cuda`, A100, `Opus 4.6`	FP16 *source-only*: 0.0254 → 0.0; FP16 *source+SASS*: 0.0254 → 0.0256	The source-only explanation notes that compiler-generated `HFMA2` materialization may exist but cannot be justified from source alone, so it conservatively predicts zero FP16 work. The SASS-backed explanation instead points to an executed `HFMA2.MMA` instruction on the taken path and counts it as real FP16 arithmetic.
Source-visible FP32 work	`boxfilter-cuda`, A100, `Opus 4.6`	FP32 *source-only*: 21.975 → 22.0	The source already exposes the 21-tap loop trip count, the 64 output threads per block, and the roughly 1 MiB unique read footprint. In this regime, source-only reasoning is already enough and post-compilation evidence adds little.
Regular stencil-like FP32 work	`asmooth-cuda`, 3080, `Opus 4.6`	FP32 *source-only*: 0.8329 → 0.8331	The kernel has a regular source-visible arithmetic and memory pattern, so the model can infer a close DRAM-level ratio without SASS.
Source/SASS-visible FP64 work	`burger-cuda`, 3080, `Opus 4.6`	FP64 *source+SASS*: 2.9669 → 2.9684	The double-precision arithmetic and global-memory structure are visible enough after lowering for the SASS-backed estimate to nearly match the profiler-derived RAI.

TABLE XI

REPEATED-QUERY RESPONSE VARIABILITY ON THE REPEAT-TRIALS PANEL. "CV" IS THE MEDIAN COEFFICIENT OF VARIATION OF PREDICTED RAI ACROSS REPEATS (LOWER IS MORE CONSISTENT); "AGR" IS THE FRACTION OF CELLS WHOSE BB/CB CALL IS UNANIMOUS ACROSS REPEATS. COMPUTED AT THE MAIN-PAPER DECODE SETTINGS.

		FP16		FP32		FP64	
Model	Evidence	CV	Agr	CV	Agr	CV	Agr
`GPT 5.4`	source-only	0.00	100%	0.21	71%	0.04	82%
`GPT 5.4`	source+SASS	0.87	86%	0.61	76%	0.33	82%
`GPT OSS`	source-only	0.00	100%	0.00	100%	0.32	50%

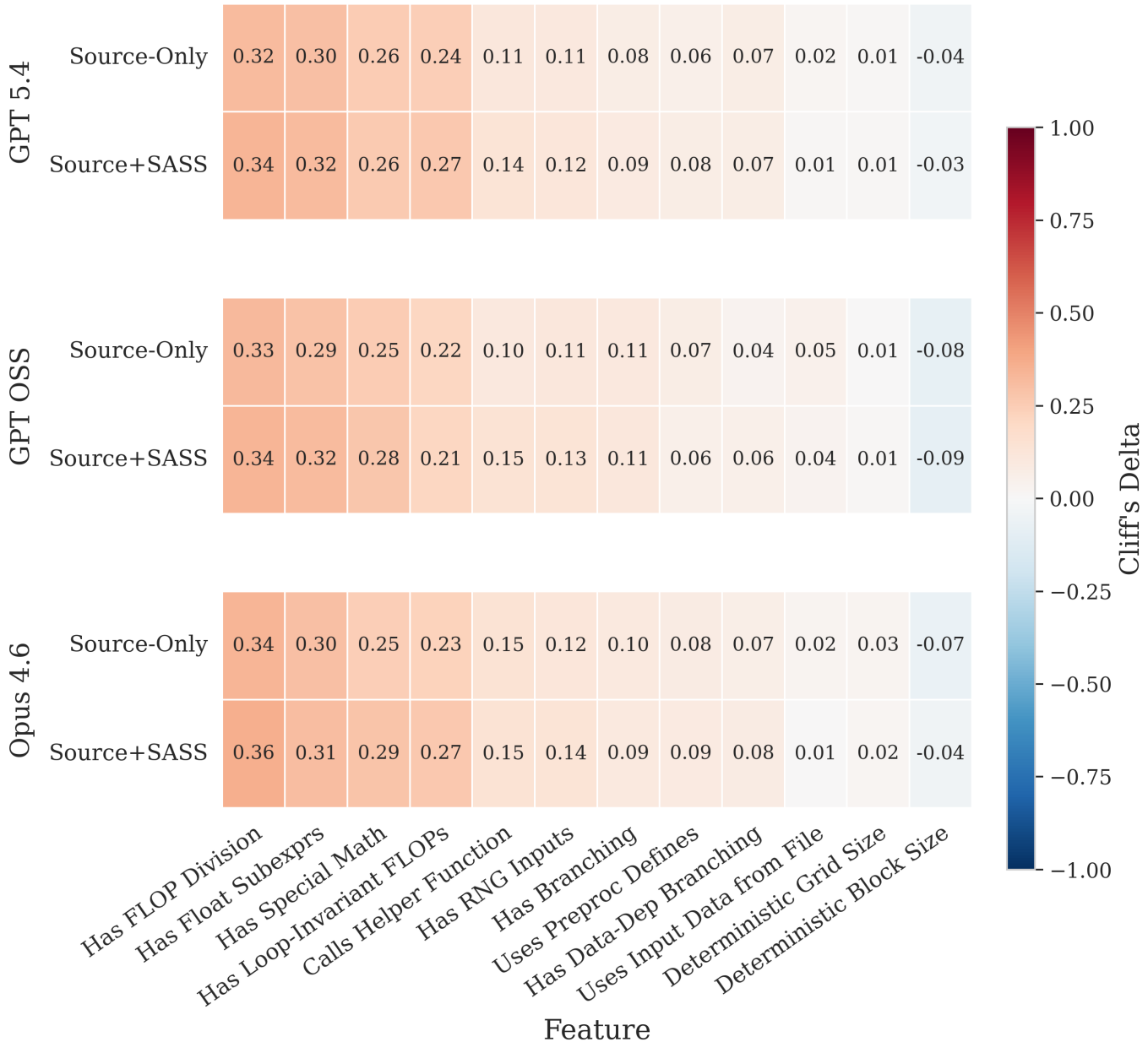

Fig. 7. Exploratory feature/error association heatmap from the released artifact. Positive Cliff's delta means higher absolute RAI error when the feature is present. The strongest repeated error signals come from division, special math, reusable subexpressions, and loop-invariant floating-point work. A post-hoc Mann–Whitney/BH sanity check across the same 72 model×prompt×feature cells preserves repeated positive associations for division, common float subexpressions, special math, loop-invariant FLOPs, helper-function calls, RNG inputs, branching, preprocessor defines, and data-dependent branching; those features appear in 7–90% of the 1,336 voted kernels. Because the feature labels are generated by an auxiliary LLM-voting pipeline, we treat this figure as descriptive rather than causal.